مراجعة نظرية (2 من 3): البرمجة غرضية التوجه (OOP)

الصف والغرض · الباني · التغليف · الوراثة · تعدد الأشكال · التجريد

1) الصف والغرض (Class and Object)

مثال من الحياة: استمارة التسجيل بالجامعة: الورقة الفارغة (فيها خانة الاسم والرقم) هي **الصف**. كل استمارة معبّأة باسم طالب هي **غرض**. نفس الورقة، قيم مختلفة.

المفهوم	ما هو	مثال
`Class`	قالب يصف الخصائص والسلوك	صف `Car` : لون، سرعة، يقود
`Object`	نسخة حقيقية مبنية من الصف بالكلمة `new`	سيارة حمراء سرعتها 120
`Attribute`	خاصية: معلومة يحملها الغرض	`color` ، `speed`
`Method`	تابع: ما يستطيع الغرض فعله	`drive()`

```java
public class Car {
    String color;
    int speed;
    void drive() {
        System.out.println(color + " car at "
+ speed);
    }
}
```

```java
Car myCar = new Car();
myCar.color = "Red";
myCar.speed = 120;
myCar.drive();
// تغيير myCar لا يؤثر على غرض آخر
```

2) الباني (Constructor) و this

الباني تابع خاص **اسمه نفس اسم الصف** وليس له نوع إرجاع. يُستدعى تلقائياً عند `new` ويعبّئ الغرض لحظة ولادته.

```java
public Student(String name, int age) {
    this.name = name;      // this.name = خاصية الغرض ، name = الوسيط
    this.age = age;
}
Student s = new Student("Sara", 19);
```

`this` = "هذا الغرض". نحتاجها عندما يتطابق اسم الوسيط مع اسم الخاصية.

3) التغليف (Encapsulation)

مثال من الحياة: الصراف الآلي: لا تمدّ يدك للصندوق، بل تضغط أزرار (سحب، إيداع) والجهاز يتحقق من الرصيد.

نخفي الخصائص بـ `private` ونسمح بالتعامل معها عبر توابع `public` تتحقق من القيم (getter / setter).

```java
private double balance;
public void deposit(double amount) {
    if (amount > 0) balance += amount;
}
public double getBalance() { return balance; }
```

الكلمة	من يستطيع الوصول
`private`	الصف نفسه فقط
(بدون كلمة)	الصفوف بنفس الحزمة (package)
`protected`	الحزمة + الصفوف الابنة
`public`	الجميع

4) الوراثة (Inheritance)

الوراثة: صف ابن يرث خصائص وتوابع صف أب بالكلمة `extends` ، ويضيف ما يخصه. العلاقة "هو" (is-a): الطالب هو شخص.

```java
public class Student extends Person {
    int id;
    public Student(String name, int age, int id) {
        super(name, age);      // استدعاء باني الأب
        this.id = id;
    }
}
```

اختبار "هو": هل الطالب شخص؟ نعم ← وراثة صحيحة. هل الشخص طالب؟ لا. فالوراثة باتجاه واحد، والأب لا يعرف توابع الابن.

5) تعدد الأشكال (Polymorphism)

	Overloading (التحميل الزائد)	Overriding (إعادة التعريف)
أين؟	داخل **نفس الصف**	بين **أب وابن**
ما الذي يتغير؟	الوسائط (عدداً أو نوعاً) والاسم نفسه	جسم التابع (الاسم والوسائط نفسها)
متى يُقرَّر؟	وقت الترجمة	وقت التشغيل (حسب الغرض الحقيقي)
مثال	`add(int,int)` و `add(double,double)`	`Cat.sound()` تعيد تعريف `Animal.sound()`

```java
int    add(int a, int b)       { return a + b;
}
double add(double a, double b) { return a + b;
}
```

```java
Animal a = new Cat();
a.sound();   // (الغرض قطة) meow
```

الفائدة: مصفوفة `Animal[]` تحوي قطة وكلباً، وحلقة واحدة `animal.sound()` تجعل كل حيوان يصدر صوته الخاص دون شرط لكل نوع.

6) التجريد (Abstraction): الصف المجرد والواجهة

	الصف المجرد `abstract class`	الواجهة `interface`
الفكرة	فكرة عامة ناقصة، لا يُبنى منها غرض	عقد من التوابع، بلا تنفيذ
مثال حياتي	"شكل": لا نرسم شكلاً بلا أن يكون دائرة أو مربعاً	مقبس الشحن: أي جهاز يطابقه يعمل
كيف نستعملها؟	`extends` (صف واحد فقط)	`implements` (واجهات كثيرة)
ما فيها؟	خصائص وتوابع عادية + مجردة	توابع بلا جسم (عقد فقط)

```
abstract class Shape { abstract void draw(); }          // لا نكتب new Shape()
class Circle extends Shape { void draw() { ... } }      // يجب تنفيذ draw
interface Drawable { void draw(); }
class Square implements Drawable, Printable { ... }     // أكثر من واجهة
```

ملخص الأفكار بجملة

المفهوم	الفكرة بجملة	كلمة Java
الصف	قالب يصف الخصائص والسلوك	**`class`**
الغرض	نسخة حقيقية من الصف	**`new`**
الباني	يعبّئ الغرض عند ولادته	**`ClassName(...)`**
التغليف	نخفي البيانات ونتحكم بها	**`private + get/set`**
الوراثة	الابن يرث من الأب (is-a)	**`extends / super`**
تعدد الأشكال	نفس الاسم، سلوك مختلف	**`overload / @Override`**
التجريد	"ماذا" بدون "كيف"	**`abstract / interface`**